

MLflow를 사용한 머신 러닝 모델 학습 및 추적 실습 (Microsoft Fabric)

이 핸즈온 실습에서는 당뇨병의 정량적 측정치를 예측하는 머신러닝 모델을 학습합니다. **Microsoft Fabric** 환경에서 **MLflow**를 활용하여 머신 러닝 모델을 학습하고 추적하는 과정을 다룹니다.

실습을 완료하면 다음을 경험할 수 있습니다:

- Microsoft Fabric의 **Notebook**, **Experiment**, **Model** 기능 활용
- **scikit-learn**으로 회귀 모델 학습
- **MLflow**로 모델 성능 비교 및 시각화

예상 소요 시간: 약 25분

※ 이 실습을 수행하려면 [Microsoft Fabric 체험 계정](#)이 필요합니다.

1단계: Workspace 생성

Microsoft Fabric에서 분석을 시작하기 전에 워크스페이스(**Workspace**)를 생성해야 합니다.

1. <https://app.fabric.microsoft.com/home?experience=fabric>에 접속하여 로그인
2. 좌측 메뉴에서 **Workspaces** (📁 모양 아이콘) 클릭
3. 새 **Workspace** 생성 → 이름 설정 후, 라이선스 모드를 **Trial** 또는 **Fabric 포함 모드**로 선택
4. 생성된 워크스페이스는 비어 있어야 합니다.

2단계: Notebook 생성

Notebook은 코드 실행, 시각화, 문서화를 동시에 할 수 있는 대화형 환경입니다.

1. 좌측 메뉴의 **Create** → **Notebook** 클릭
(Create가 보이지 않으면 ... 버튼을 눌러 확장)
2. 노트북 이름 지정
3. 첫 셀을 **Markdown** 셀로 전환 → 아래 텍스트 입력

```
# Train a machine learning model and track with MLflow
```

3단계: 데이터 로딩 (Azure Open Dataset)

Azure Open Dataset 중 하나인 당뇨병(**diabetes**) 데이터셋을 불러옵니다.

1. 코드 셀 추가 후 아래 코드 입력 및 실행:

```
# Azure Blob Storage 설정
blob_account_name = "azureopendatastorage"
blob_container_name = "mlsamples"
blob_relative_path = "diabetes"
blob_sas_token = r"" # 공개 접근

# 경로 설정 및 Spark 구성
wasbs_path =
f"wasbs://{blob_container_name}@{blob_account_name}.blob.core.windows.net/{blob_relative_p
ath}"
spark.conf.set(f"fs.azure.sas.{blob_container_name}.
{blob_account_name}.blob.core.windows.net", blob_sas_token)
print("Remote blob path:", wasbs_path)

# Parquet 형식 데이터 로딩
df = spark.read.parquet(wasbs_path)
```

2. 다음 셀 추가 → 데이터 프리뷰

```
display(df)
```

출력은 당뇨병 데이터셋의 행과 열을 보여줍니다. **y** 열이 우리가 예측하려는 목표 변수(당뇨병 진행도의 정량적 측정치)입니다.

3. Pandas 형식으로 변환 (scikit-learn 학습용)

scikit-learn 라이브러리는 Pandas 데이터프레임 형식의 입력을 기대합니다. 아래 코드를 실행하여 데이터셋을 Pandas 데이터프레임으로 변환합니다.

```
import pandas as pd
df = df.toPandas()
df.head()
```

4단계: 모델 학습

데이터를 로드했으므로, 이제 이를 사용하여 머신러닝 모델을 학습하고 당뇨병의 정량적 측정치를 예측할 수 있습니다. scikit-learn 라이브러리를 사용하여 회귀 모델을 학습하고 MLflow로 모델을 추적할 것입니다.

개념 설명: MLflow란?

MLflow는 머신러닝 수명주기(lifecycle)를 관리하기 위한 오픈소스 플랫폼입니다. 복잡한 머신러닝 프로젝트에서 다음과 같은 주요 문제를 해결하는 데 도움을 줍니다.

실험 추적 (Tracking): 어떤 데이터, 코드, 파라미터를 사용해서 모델을 만들었는지 모든 정보를 기록합니다. 이를 통해 실험을 재현하고 결과를 비교하기가 매우 쉬워집니다.

모델 패키징 (Packaging): 학습된 모델을 재사용 가능한 형식으로 패키징합니다.

모델 배포 (Deployment): 패키징된 모델을 다양한 환경에 배포하는 것을 도와줍니다.

모델 레지스트리 (Registry): 버전 관리, 스테이징(staging), 프로덕션(production) 전환 등 모델을 체계적으로 관리하는 중앙 저장소 역할을 합니다.

1. 데이터 분할

```
from sklearn.model_selection import train_test_split

x = df[['AGE', 'SEX', 'BMI', 'BP', 'S1', 'S2', 'S3', 'S4', 'S5', 'S6']].values
y = df['Y'].values

x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=0)
```

코드 설명

- from sklearn.model_selection import train_test_split: `scikit-learn` 라이브러리에서 데이터 분할을 위한 함수를 가져옵니다.
- 데이터프레임 df에서 입력값(독립변수, 특성, features)으로 사용할 열(columns) 10개를 선택합니다.
- 이 열들은 환자의 나이, 성별, 혈압, 콜레스테롤 수치 등 당뇨와 관련된 생체 정보입니다.
- .values 속성을 사용해 Pandas DataFrame이 아닌 **NumPy 배열(numpy.ndarray)**로 변환합니다.→ scikit-learn은 대부분 NumPy 배열을 사용합니다.
- 파이썬 문법: df[['col1', 'col2']] 형태는 다중 열을 선택하는 방법입니다.
- .values는 Pandas에서 데이터를 NumPy 형태로 꺼낼 때 사용합니다.
- `x, y = ...`: 데이터를 두 부분으로 나눕니다. `x`는 모델이 예측을 위해 사용할 입력 변수인 피쳐들입니다(AGE, SEX, BMI 등). `y`는 모델이 예측해야 할 목표 값인 레이블(`y`, 당뇨병 진행도)입니다.
- `train_test_split(...)`: 전체 데이터를 학습 데이터와 테스트 데이터로 나눕니다. 모델은 학습 데이터(`x_train`, `y_train`)를 보고 패턴을 학습하며, 테스트 데이터(`x_test`, `y_test`)는 모델이 한 번도 보지 못한 새로운 데이터에 대해 얼마나 잘 작동하는지 성능을 평가하는 데 사용됩니다. `test_size=0.30`은 데이터의 30%를 테스트용으로 사용하겠다는 의미입니다. `random_state=0`은 실행할 때마다 동일한 방식으로 데이터를 분할하여 재현성을 보장합니다.

2. Experiment 설정

```
import mlflow
experiment_name = "experiment-diabetes"
mlflow.set_experiment(experiment_name)
```

코드 설명

- import mlflow: MLflow 라이브러리를 가져옵니다.
- `mlflow.set_experiment(experiment_name)`: MLflow에서 관련 있는 실행(Run)들을 그룹화하는 단위를 Experiment라고 합니다. 이 코드는 `"experiment-diabetes"` 라는 이름의 Experiment를 활성화합니다. 만약 이 이름의 Experiment가 없다면 새로 생성하고, 있다면 기존의 것을 사용합니다. 앞으로 학습할 모델들의 모든 정보는 이 Experiment 안에 기록됩니다.

3. 선형 회귀 모델 학습

```
from sklearn.linear_model import LinearRegression

with mlflow.start_run():
    mlflow.autolog()
    model = LinearRegression()
    model.fit(X_train, y_train)
    mlflow.log_param("estimator", "LinearRegression")
```

모델 및 코드 설명

- **모델: 선형 회귀 (Linear Regression):** 가장 기본적인 회귀 알고리즘 중 하나입니다. 피쳐들과 목표 변수 사이에 직선 관계가 있다고 가정하고, 데이터에 가장 잘 맞는 직선(또는 초평면)을 찾습니다. 모델이 단순하고 해석하기 쉬워 베이스라인 모델로 자주 사용됩니다.
- `with mlflow.start_run():`: 이 블록 안에서 실행되는 모든 코드는 하나의 MLflow **Run**(실행)으로 간주되어 기록됩니다.
- `mlflow.autolog()`: **MLflow의 매우 강력한 기능입니다.** 이 한 줄의 코드는 `scikit-learn` 과 같은 일반적인 라이브러리에 대해 모델의 하이퍼파라미터(hyperparameter), 성능 지표(metric), 그리고 학습된 모델 아티팩트(artifact)까지 **자동으로** 기록해 줍니다. 수동으로 `mlflow.log_metric()` 이나 `mlflow.log_param()` 을 여러 번 호출할 필요가 없어 코드가 매우 간결해집니다.
- `model.fit(X_train, y_train)`: `LinearRegression` 모델을 학습 데이터로 학습시킵니다.
- `mlflow.log_param("estimator", "LinearRegression")`: `autolog` 가 많은 것을 기록해주지만, 우리가 직접 추가 정보를 기록하고 싶을 때 사용합니다. 여기서는 나중에 모델을 쉽게 구분하기 위해 "estimator"라는 이름의 파라미터를 "LinearRegression"이라는 값으로 직접 기록합니다.

4. 결정 트리 회귀 모델 학습

```
from sklearn.tree import DecisionTreeRegressor

with mlflow.start_run():
    mlflow.autolog()
    model = DecisionTreeRegressor(max_depth=5)
    model.fit(X_train, y_train)
    mlflow.log_param("estimator", "DecisionTreeRegressor")
```

모델 및 코드 설명

- **모델: 결정 트리 회귀 (Decision Tree Regressor):** 데이터를 특정 기준(예: 'BMI > 25인가?')에 따라 반복적으로 분할하는 '스무고개'와 같은 방식으로 예측을 수행합니다. 선형 회귀와 달리 비선형 관계도 잘 포착할 수 있습니다. `max_depth=5` 는 트리의 깊이를 5단계로 제한하는 하이퍼파라미터로, 모델이 과도하게 복잡해져 학습 데이터에만 과적합(overfitting)되는 것을 방지합니다.
- 코드 구조는 이전 `LinearRegression` 모델 학습과 동일합니다. `mlflow.autolog()` 가 `DecisionTreeRegressor` 모델의 `max_depth` 와 같은 파라미터와 성능 지표를 자동으로 기록하고, 우리는 "estimator" 파라미터를 "DecisionTreeRegressor"로 명시적으로 기록하여 두 모델 실행을 구분합니다.

5단계: MLflow로 실험 조회 및 비교

MLflow로 모델을 학습하고 추적했다면, 이제 MLflow 라이브러리를 사용하여 프로그래밍 방식으로 실험과 그 세부 정보를 검색할 수 있습니다.

실험 목록 조회

```
experiments = mlflow.search_experiments()
for exp in experiments:
    print(exp.name)
```

실험 및 실행 정보 조회

특정 실험을 검색하려면 이름으로 가져올 수 있습니다.

```
exp = mlflow.get_experiment_by_name("experiment-diabetes")
print(exp)
mlflow.search_runs(exp.experiment_id)
```

최근 실행 2개만 비교

실행 결과를 더 쉽게 비교하기 위해 검색 결과를 정렬하도록 구성할 수 있습니다. 예를 들어, 다음 셀은 결과를 start_time 기준으로 내림차순 정렬하고 최대 2개의 결과만 보여줍니다.

```
mlflow.search_runs(exp.experiment_id, order_by=["start_time DESC"], max_results=2)
```

모델별 R2 score 시각화

```
import matplotlib.pyplot as plt

df_results = mlflow.search_runs(
    exp.experiment_id, order_by=["start_time DESC"], max_results=2
)[["metrics.training_r2_score", "params.estimator"]]

fig, ax = plt.subplots()
ax.bar(df_results["params.estimator"], df_results["metrics.training_r2_score"])
ax.set_xlabel("Estimator")
ax.set_ylabel("R2 score")
ax.set_title("R2 score by Estimator")

for i, v in enumerate(df_results["metrics.training_r2_score"]):
    ax.text(i, v, str(round(v, 2)), ha='center', va='bottom', fontweight='bold')

plt.show()
```

6단계: Fabric에서 실험 시각적으로 비교

1. 왼쪽 메뉴 바에서 Workspace로 이동합니다.

2. experiment-diabetes 실험을 선택하여 엽니다.
- 팁: 기록된 실험 실행이 보이지 않으면 페이지를 새로고침하세요.
3. View 탭이 선택되어 있는지 확인하고 Run list를 선택합니다.
4. 각각의 확인란을 선택하여 가장 최근의 두 실행을 선택합니다.
5. 두 실행을 선택하면, 화면 하단에 Metric comparison 창이 나타나 두 실행의 결과를 서로 비교할 수 있습니다. 기본적으로 메트릭은 실행 이름별로 표시됩니다.
6. 각 실행의 평균 제곱 오차(MSE) 또는 R2 점수를 시각화하는 그래프의 □ (Edit) 버튼을 선택합니다.
7. visualization type을 bar로 변경합니다.
8. X-axis를 estimator로 변경합니다.
9. Replace를 선택하고 새로운 그래프를 탐색합니다.

7단계: 최종 모델 저장

실험 실행 중에 가장 성능이 좋은 모델을 찾았다면, 이 모델을 "저장"하여 나중에 예측에 사용할 수 있습니다. MLflow에서 모델을 저장하는 것은 단순히 모델 파일을 저장하는 것을 넘어, Fabric 내의 모델 레지스트리에 모델을 "등록"하는 것을 의미합니다. 모델 레지스트리는 모델의 버전 관리, 상태(예: Staging, Production) 관리, 설명 추가 등 모델을 체계적으로 관리하는 중앙 저장소 역할을 합니다. 이를 통해 어떤 버전의 모델이 어떤 용도로 사용되고 있는지 쉽게 추적하고 관리할 수 있습니다.

실험 실행 전반에 걸쳐 학습한 머신러닝 모델을 비교한 후, 가장 성능이 좋은 모델을 선택할 수 있습니다. 가장 성능이 좋은 모델을 사용하려면 모델을 저장하고 이를 사용하여 예측을 생성합니다.

1. 실험 화면에서 **View > Run details**
2. **Training R2 score**가 가장 높은 실행 항목 선택
3. 오른쪽에서 **Save run as model** 선택
4. **Create new model > model-diabetes** 저장
5. 모델이 생성되면 화면 오른쪽 상단에 나타나는 알림에서 View ML model을 선택합니다. 또는 창을 새로고침하여 Workspace의 Models 목록에서 확인할 수도 있습니다.

8단계: 세션 종료 및 정리

1. 노트북 상단 메뉴에서  **Settings** 클릭 → 이름: `Train and compare models` 로 변경
2. 메뉴에서 **Stop session** 선택
3. 필요 시 좌측 ... > **Workspace settings > Remove this workspace**로 삭제

주요 용어 정리

용어	설명
Notebook	코드 작성, 실행, 시각화를 동시에 할 수 있는 인터랙티브 환경
MLflow	머신러닝 실험 관리 도구. 모델 성능 추적, 비교 가능
Experiment	MLflow에서 모델 학습 세션을 구분하는 단위
Run	하나의 모델 학습 실행 과정
Artifact	학습 결과로 저장된 파일 (모델 파일, 그래프 등)

이 실습은 Microsoft Fabric에서 데이터 사이언스의 전체 흐름을 체험하도록 합니다.
특히, 모델 학습 후 비교 및 선택, **MLflow** 기반 실험 관리, **GUI** 기반 모델 등록 등은
실무에서도 유용한 패턴입니다.